Safe Controller for Planar Multirotor

Hyeonjae Park*
Georgia Institute of Technology

April 29, 2026

Abstract

This report presents the design and simulation of progressively advanced controllers for a planar multirotor model. Starting from a linearized hover controller, we develop a dynamic feedback linearization scheme for trajectory tracking and incorporate Control Barrier Functions (CBFs) to enforce safety constraints. Each stage is derived analytically and validated through simulation.

1 Problem Description

We consider a planar multirotor, a 2D simplification of a quadrotor maneuvering in a vertical plane. The horizontal position is x_1, the vertical position is x_2, and the roll angle is θ. The two control inputs are the net thrust u_1 through the center of gravity in the direction $[-\sin\theta,\ \cos\theta]^T$, and the rolling moment u_2. The equations of motion are

$$m\ddot{x}_1 = -u_1 \sin\theta, \tag{1}$$
$$m\ddot{x}_2 = u_1 \cos\theta - mg, \tag{2}$$
$$I\ddot{\theta} = u_2, \tag{3}$$

where we take $m = 1$, $I = 1$, and $g = 9.8$ throughout.

2 Part I: Modeling and Hover

2.1 State-Space Formulation

We introduce the velocity states $v_1 = \dot{x}_1$, $v_2 = \dot{x}_2$, and $\omega = \dot{\theta}$, and define the state and input vectors

$$x = \begin{bmatrix} x_1 & x_2 & v_1 & v_2 & \theta & \omega \end{bmatrix}^T, \quad u = \begin{bmatrix} u_1 & u_2 \end{bmatrix}^T.$$

Substituting $m = I = 1$ and $g = 9.8$ into (1)–(3), the equations of motion simplify to

$$\ddot{x}_1 = -u_1 \sin\theta, \tag{4}$$
$$\ddot{x}_2 = u_1 \cos\theta - 9.8, \tag{5}$$
$$\ddot{\theta} = u_2. \tag{6}$$

To convert these second-order equations into first-order form, we use the definitions $v_1 = \dot{x}_1$, $v_2 = \dot{x}_2$, and $\omega = \dot{\theta}$. Each second-order equation $\ddot{q} = a$ is split into two first-order equations: $\dot{q} = \dot{q}$ (kinematic) and $\frac{d}{dt}\dot{q} = a$ (dynamic). Applying this to each degree of freedom:

Horizontal (x_1):

$$\dot{x}_1 = v_1, \tag{7}$$
$$\dot{v}_1 = \ddot{x}_1 = -u_1 \sin\theta. \tag{8}$$

*The complete code for the simulations and figures presented in this report is publicly available at https://github.com/hjaepark/Safe-Controller-for-Planar-Multirotor.

Vertical (x_2):

$$\dot{x}_2 = v_2, \tag{9}$$
$$\dot{v}_2 = \ddot{x}_2 = u_1 \cos\theta - 9.8. \tag{10}$$

Rotation (θ):

$$\dot{\theta} = \omega, \tag{11}$$
$$\dot{\omega} = \ddot{\theta} = u_2. \tag{12}$$

Collecting all six first-order equations into control-affine form yields the state-space representation:

$$\dot{x} = f(x) + g_1(x)\,u_1 + g_2(x)\,u_2 = \underbrace{\begin{bmatrix} v_1 \\ v_2 \\ 0 \\ -9.8 \\ \omega \\ 0 \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0 \\ 0 \\ -\sin\theta \\ \cos\theta \\ 0 \\ 0 \end{bmatrix}}_{g_1(x)} u_1 + \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}}_{g_2(x)} u_2. \tag{13}$$

The drift term $f(x)$ captures the natural dynamics (gravity and kinematics), while $g_1(x)$ and $g_2(x)$ describe how each input enters the system.

2.2 Simulator

The state-space model (13) was implemented in Python using `scipy.integrate.odeint` for numerical integration. To verify the simulator, we simulate with the initial condition $x_0 = [0, 2, 10, 10, 0, 0]^T$ and zero control input ($u_1 = u_2 = 0$). Under no control, the multirotor follows a ballistic trajectory: x_1 increases linearly (constant horizontal velocity), x_2 traces a parabola (free-fall under gravity), and θ remains constant. The resulting trajectories, shown in Figure 1, match the expected behavior.

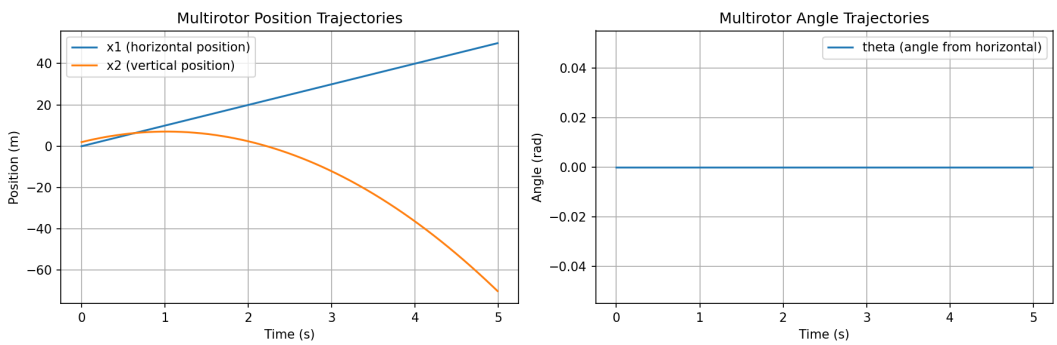

Figure 1: Verification of the simulator with $x_0 = [0, 2, 10, 10, 0, 0]^T$ and no control input.

2.3 Equilibrium Analysis

To find the equilibrium state x^* and input u^* for hovering at a desired position (x_1^*, x_2^*), we set $\dot{x} = 0$ in (13) and solve each equation. From the first, second, and fifth rows, we immediately obtain the kinematic conditions:

$$v_1^* = 0, \tag{14}$$
$$v_2^* = 0, \tag{15}$$
$$\omega^* = 0. \tag{16}$$

These state that all velocities must vanish at hover, which is physically intuitive. From the sixth row, the input condition $u_2^* = 0$ follows directly, meaning no torque is required to maintain a constant

orientation. The third and fourth rows must be considered simultaneously:

$$-u_1^* \sin\theta^* = 0, \tag{17}$$

$$u_1^* \cos\theta^* - 9.8 = 0. \tag{18}$$

From (18), the thrust u_1^* cannot be zero (otherwise $0 = 9.8$, a contradiction). Therefore, (17) requires $\sin\theta^* = 0$, giving $\theta^* = 0$. Substituting back into (18) yields $u_1^* = 9.8$, meaning the thrust exactly cancels gravity ($mg = 1 \times 9.8$). Finally, note that x_1 and x_2 do not appear on the right-hand side of any equation in (13). This means the equilibrium conditions impose no constraint on position, the multirotor can hover at any (x_1^*, x_2^*). The equilibrium state and input are therefore:

$$x^* = \begin{bmatrix} x_1^* & x_2^* & 0 & 0 & 0 & 0 \end{bmatrix}^T, \quad u^* = \begin{bmatrix} 9.8 \\ 0 \end{bmatrix}. \tag{19}$$

2.4 Linearization

To design a linear controller, we linearize the nonlinear model (13) about the equilibrium (19). Defining the perturbation variables $\tilde{x} = x - x^*$ and $\tilde{u} = u - u^*$, a first-order Taylor expansion about (x^*, u^*) gives

$$\dot{\tilde{x}} = A\tilde{x} + B\tilde{u}, \tag{20}$$

where the Jacobian matrices are

$$A = \left.\frac{\partial(f + g\,u)}{\partial x}\right|_{x^*,u^*}, \quad B = \left.\frac{\partial(f + g\,u)}{\partial u}\right|_{x^*,u^*}. \tag{21}$$

Note that the zeroth-order term vanishes by definition of the equilibrium, and higher-order terms are neglected under the assumption that $\tilde{x}$ and $\tilde{u}$ remain small.

We compute A by taking the partial derivative of each component of $f(x) + g(x)u$ with respect to each state variable, then evaluating at $\theta^* = 0$ and $u_1^* = 9.8$. This yields:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -9.8 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}. \tag{22}$$

The $(3, 5)$ entry -9.8 has a clear physical interpretation: a small positive perturbation in θ tilts the thrust vector, producing a horizontal acceleration proportional to the equilibrium thrust $u_1^* = 9.8$. The $(4, 5)$ entry is zero because $\partial f_4/\partial\theta = -u_1 \sin\theta$ vanishes at $\theta^* = 0$, meaning a small tilt does not affect the vertical acceleration to first order.

For B, we differentiate with respect to $u = [u_1, u_2]^T$. Only the third, fourth, and sixth rows depend on the inputs: $\partial(\cdot)/\partial u_1 = [-\sin\theta, \cos\theta]$ and $\partial(\cdot)/\partial u_2 = 1$. Evaluating at $\theta^* = 0$:

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}. \tag{23}$$

The structure of B reveals that, at hover, the thrust perturbation $\tilde{u}_1$ directly affects only the vertical acceleration (row 4), while the torque u_2 directly affects only the angular acceleration (row 6). The horizontal acceleration is not directly controllable through the inputs at this equilibrium. It can only be influenced indirectly by first tilting the multirotor (changing θ), which then produces a horizontal force. This underactuation is a key characteristic of the planar multirotor.

2.5 Linear Hover Controller

With the linearized model $\dot{\tilde{x}} = A\tilde{x} + B\tilde{u}$, we design a state-feedback controller of the form $\tilde{u} = K\tilde{x}$, which yields the closed-loop dynamics $\dot{\tilde{x}} = (A+BK)\tilde{x}$. The goal is to choose K such that all eigenvalues of $A + BK$ have strictly negative real parts, guaranteeing asymptotic stability of the equilibrium. We use the Linear Quadratic Regulator (LQR) to compute K. LQR selects the gain that minimizes the infinite-horizon cost function

$$J = \int_0^\infty \left(\tilde{x}^T Q \tilde{x} + \tilde{u}^T R \tilde{u}\right) dt, \tag{24}$$

where $Q \in \mathbb{R}^{6\times 6}$ penalizes state error and $R \in \mathbb{R}^{2\times 2}$ penalizes input effort. The optimal gain is given by $K = -R^{-1}B^T P$, where P is the unique positive-definite solution to the continuous-time algebraic Riccati equation

$$A^T P + PA - PBR^{-1}B^T P + Q = 0. \tag{25}$$

We choose $Q = \mathrm{diag}(10, 10, 1, 1, 10, 1)$ and $R = \mathrm{diag}(1, 1)$, placing higher penalties on position and angle errors to prioritize fast convergence in these states. Solving the Riccati equation yields the gain

$$K = \begin{bmatrix} 0 & -3.16 & 0 & -2.71 & 0 & 0 \\ 3.16 & 0 & 3.80 & 0 & -20.86 & -6.54 \end{bmatrix}. \tag{26}$$

The full control law applied to the nonlinear system is then $u = u^* + K(x - x^*)$, where x^* and u^* are the equilibrium state and input from (19). Since the matrices A and B do not depend on the hover position (x_1^*, x_2^*), the same gain K is valid for any desired hover location. The structure of K reflects the underactuation discussed in the linearization analysis. The first row (thrust correction) contains nonzero entries only for x_2 and v_2, confirming that thrust adjustments directly address only vertical errors. The second row (torque) couples horizontal position and velocity errors into the rolling moment, implementing the indirect control path: horizontal error is corrected by first tilting the multirotor via torque, which redirects the thrust vector to produce horizontal motion.

We validate the controller on the full nonlinear model for three hover positions: $(0, 5)$, $(3, 10)$, and $(-2, 7)$, each with an initial condition offset from the target. As shown in Figure 2, all positions converge to their respective targets within approximately 5 seconds, and the roll angle returns to zero. The transient roll excursions of up to 30° arise from the underactuated coupling, as the controller must tilt the multirotor to correct horizontal position errors. Figure 3 demonstrates robustness to process noise (Gaussian, $\sigma = 0.5$ on acceleration states), where the controller maintains stability with small steady-state fluctuations.

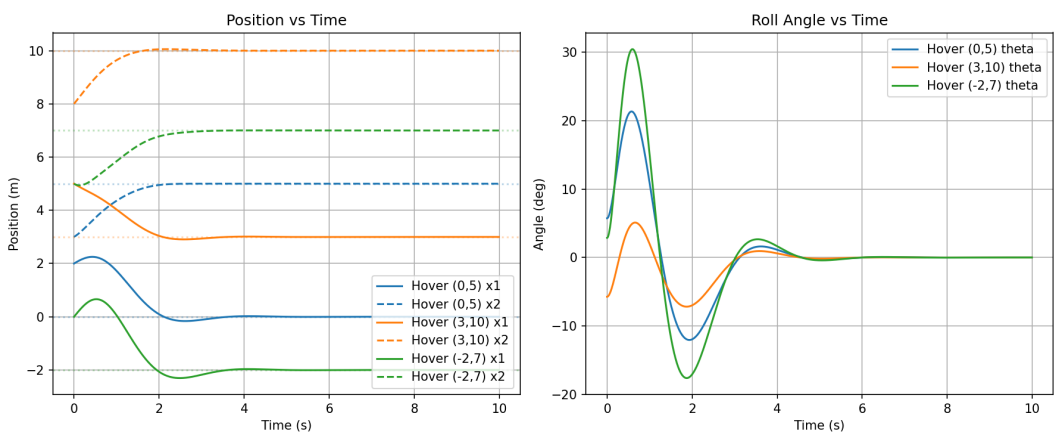

Figure 2: LQR hover controller without noise. Positions converge to the desired hover targets (dotted lines) for three different locations. The roll angle exhibits transient oscillations due to underactuated coupling before settling to zero.

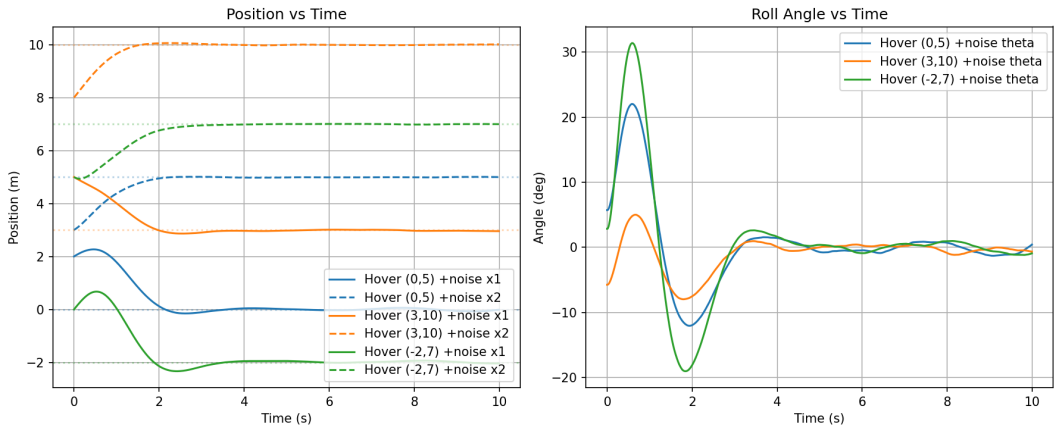

Figure 3: LQR hover controller with Gaussian process noise ($\sigma = 0.5$). The controller maintains stability with small steady-state fluctuations around the target positions.

3 Part II: Dynamic Feedback Linearization (DFL)

The hover controller from Part I stabilizes the multirotor at a fixed point, but its design relies on two assumptions that limit its applicability. First, the linearization is valid only in a neighborhood of the equilibrium $\theta^* = 0$, where the small-angle approximations $\sin\theta \approx \theta$ and $\cos\theta \approx 1$ hold. Large excursions in roll, such as those required for aggressive maneuvers, violate this assumption and degrade the closed-loop behavior. Second, the LQR formulation is intrinsically a regulation problem, not a tracking problem. the equilibrium state X^* is constant, and the controller has no mechanism to follow a time-varying reference $y^*(t)$. To track trajectories such as a circle or a figure-eight, we need a controller that handles the full nonlinear dynamics globally and accommodates time-varying setpoints. In this section, we develop such a controller using dynamic feedback linearization, which exactly cancels the nonlinearities through a coordinate transformation and an inversion of the input map.

3.1 Relative Degree Analysis

To determine whether the system is feedback linearizable, we take the output functions $h_1(x) = x_1$ and $h_2(x) = x_2$, with $y = \begin{bmatrix} h_1 & h_2 \end{bmatrix}^T$, and compute successive Lie derivatives along f and g.

For the first output $h_1(x) = x_1$:

$$\dot{y}_1 = L_f h_1 = v_1, \tag{27}$$

$$\ddot{y}_1 = L_f^2 h_1 + L_{g_1} L_f h_1 \cdot u_1 + L_{g_2} L_f h_1 \cdot u_2 = -u_1 \sin\theta. \tag{28}$$

The input u_1 appears at the second derivative, but it is multiplied by $\sin\theta$, which vanishes at the hover equilibrium $\theta^* = 0$. The input u_2 does not appear, i.e., $L_{g_2} L_f h_1 = 0$.

For the second output $h_2(x) = x_2$:

$$\dot{y}_2 = L_f h_2 = v_2, \tag{29}$$

$$\ddot{y}_2 = L_f^2 h_2 + L_{g_1} L_f h_2 \cdot u_1 + L_{g_2} L_f h_2 \cdot u_2 = u_1 \cos\theta - 9.8. \tag{30}$$

Here u_1 appears multiplied by $\cos\theta$, which is nonzero at hover. Again, $L_{g_2} L_f h_2 = 0$.

For a MIMO system, a well-defined vector relative degree requires the decoupling matrix to be invertible. At the second derivative, this matrix is

$$A(x) = \begin{bmatrix} L_{g_1} L_f h_1(x) & L_{g_2} L_f h_1(x) \\ L_{g_1} L_f h_2(x) & L_{g_2} L_f h_2(x) \end{bmatrix} = \begin{bmatrix} -\sin\theta & 0 \\ \cos\theta & 0 \end{bmatrix}. \tag{31}$$

The second column is identically zero for all θ, since $L_{g_2} L_f h_i = 0$ for $i = 1, 2$. Therefore $\det(A) = 0$ regardless of the state, and $A(x)$ is always singular. Physically, this occurs because u_2 (torque) affects

the positions x_1 and x_2 only indirectly: u_2 drives $\ddot{\theta}$, which changes θ, which then redirects the thrust vector to influence $\ddot{x}_1$ and $\ddot{x}_2$. Two Lie derivatives are not sufficient to reveal this coupling path. Since $A(x)$ is not invertible, the system does not have a well-defined vector relative degree $\{2, 2\}$, and standard input-output feedback linearization cannot be applied.

3.2 Dynamic Extension

Since the original system lacks a well-defined vector relative degree, we introduce auxiliary states by appending a chain of integrators to the input u_1. Define:

$$x_7 = u_1, \quad \dot{x}_7 = x_8, \quad \dot{x}_8 = \tilde{u}_1, \tag{32}$$

where $\tilde{u}_1$ is the new control input. This creates the integrator chain

$$\tilde{u}_1 \xrightarrow{\int} x_8 \xrightarrow{\int} x_7 = u_1, \tag{33}$$

so that the original thrust u_1 is now a state rather than a direct input. The extended state vector becomes $x_{\text{ext}} = [x_1,\, x_2,\, v_1,\, v_2,\, \theta,\, \omega,\, x_7,\, x_8]^T \in \mathbb{R}^8$, and the new input vector is $[\tilde{u}_1,\, u_2]^T$. Substituting $x_7 = u_1$ into the original dynamics, the extended system is

$$\dot{x}_{\text{ext}} = \begin{bmatrix} v_1 \\ v_2 \\ -x_7 \sin\theta \\ x_7 \cos\theta - 9.8 \\ \omega \\ u_2 \\ x_8 \\ \tilde{u}_1 \end{bmatrix}. \tag{34}$$

We now repeat the relative degree analysis on this extended system. Differentiating $y_1 = x_1$ successively:

$$\dot{y}_1 = v_1, \tag{35}$$
$$\ddot{y}_1 = -x_7 \sin\theta, \tag{36}$$
$$y_1^{(3)} = -x_8 \sin\theta - x_7\omega \cos\theta, \tag{37}$$
$$y_1^{(4)} = -\tilde{u}_1 \sin\theta - x_7 u_2 \cos\theta + b_1(x_{\text{ext}}), \tag{38}$$

where $b_1(x_{\text{ext}})$ collects terms that depend only on states. The new inputs $\tilde{u}_1$ and u_2 first appear together at the fourth derivative. Similarly, differentiating $y_2 = x_2$:

$$\dot{y}_2 = v_2, \tag{39}$$
$$\ddot{y}_2 = x_7 \cos\theta - 9.8, \tag{40}$$
$$y_2^{(3)} = x_8 \cos\theta - x_7\omega \sin\theta, \tag{41}$$
$$y_2^{(4)} = \tilde{u}_1 \cos\theta - x_7 u_2 \sin\theta + b_2(x_{\text{ext}}). \tag{42}$$

Both inputs appear simultaneously at the fourth derivative of each output. The new decoupling matrix is

$$A_{\text{ext}}(x) = \begin{bmatrix} -\sin\theta & -x_7 \cos\theta \\ \cos\theta & -x_7 \sin\theta \end{bmatrix}, \tag{43}$$

with determinant

$$\det(A_{\text{ext}}) = (-\sin\theta)(-x_7 \sin\theta) - (-x_7 \cos\theta)(\cos\theta) = x_7(\sin^2\theta + \cos^2\theta) = x_7. \tag{44}$$

Since $x_7 = u_1$ and the equilibrium thrust is $u_1^* = 9.8 \neq 0$, the decoupling matrix is invertible in any operating regime where the thrust is nonzero. The extended system therefore has a well-defined vector relative degree $\{4,\, 4\}$, and the total relative degree $r = 4 + 4 = 8$ equals the dimension of the

extended state space $n = 8$. The coordinate transformation for the extended system maps the state x_{ext} into input-output coordinates. Since $r = n = 8$, there are no zero dynamics coordinates, and the full transformation is

$$T : x_{\text{ext}} \mapsto \begin{bmatrix} \eta_1 \\ \eta_2 \\ \eta_3 \\ \eta_4 \\ \eta_5 \\ \eta_6 \\ \eta_7 \\ \eta_8 \end{bmatrix} = \begin{bmatrix} y_1 \\ \dot{y}_1 \\ \ddot{y}_1 \\ y_1^{(3)} \\ y_2 \\ \dot{y}_2 \\ \ddot{y}_2 \\ y_2^{(3)} \end{bmatrix} = \begin{bmatrix} x_1 \\ v_1 \\ -x_7 \sin\theta \\ -x_8 \sin\theta - x_7\omega\cos\theta \\ x_2 \\ v_2 \\ x_7\cos\theta - 9.8 \\ x_8\cos\theta - x_7\omega\sin\theta \end{bmatrix}. \tag{45}$$

All eight states are expressed as successive derivatives of the two outputs, leaving no internal dynamics unaccounted for.

3.3 Zero Dynamics

As shown in the preceding section, the total relative degree of the extended system is $r = 4 + 4 = 8$, which equals the dimension of the extended state space $n = 8$. The coordinate transformation (45) maps all eight states into output derivatives, leaving no internal dynamics unaccounted for. Therefore, the zero dynamics are trivially empty, and the system is minimum phase.

3.4 Tracking Controller Design

With the well-defined vector relative degree $\{4, 4\}$ and invertible decoupling matrix $A_{\text{ext}}(x)$, we can now design a feedback linearizing controller for trajectory tracking. From (38) and (42), the fourth derivatives of the outputs can be written compactly as

$$\begin{bmatrix} y_1^{(4)} \\ y_2^{(4)} \end{bmatrix} = \underbrace{\begin{bmatrix} b_1(x_{\text{ext}}) \\ b_2(x_{\text{ext}}) \end{bmatrix}}_{b(x)} + A_{\text{ext}}(x) \begin{bmatrix} \tilde{u}_1 \\ u_2 \end{bmatrix}. \tag{46}$$

Since $A_{\text{ext}}(x)$ is invertible whenever $x_7 \neq 0$, we apply the feedback linearizing control law

$$\begin{bmatrix} \tilde{u}_1 \\ u_2 \end{bmatrix} = A_{\text{ext}}^{-1}(x)\,(v - b(x)), \tag{47}$$

where $v = [v_1,\, v_2]^T$ is a virtual input to be designed. Substituting (47) into (46) cancels all nonlinear terms, yielding two decoupled linear systems:

$$y_1^{(4)} = v_1, \quad y_2^{(4)} = v_2. \tag{48}$$

Each output channel is now an independent chain of four integrators. To track a desired trajectory $y^*(t) = [y_1^*(t),\, y_2^*(t)]^T$, we define the tracking errors $e_i = y_i - y_i^*$ for $i = 1, 2$ and choose v_i to stabilize the error dynamics:

$$v_i = y_i^{*(4)} - k_3\left(y_i^{(3)} - y_i^{*(3)}\right) - k_2\,(\ddot{y}_i - \ddot{y}_i^*) - k_1\,(\dot{y}_i - \dot{y}_i^*) - k_0\,(y_i - y_i^*). \tag{49}$$

Substituting into (48) yields the closed-loop error dynamics for each channel:

$$e_i^{(4)} + k_3\, e_i^{(3)} + k_2\,\ddot{e}_i + k_1\,\dot{e}_i + k_0\, e_i = 0. \tag{50}$$

This is a linear, constant-coefficient ODE whose stability is determined by the characteristic polynomial

$$s^4 + k_3\, s^3 + k_2\, s^2 + k_1\, s + k_0 = 0. \tag{51}$$

We place all four roots at $s = -\lambda$ by choosing the gains from the binomial expansion $(s + \lambda)^4 = s^4 + 4\lambda\, s^3 + 6\lambda^2\, s^2 + 4\lambda^3\, s + \lambda^4$, giving

$$k_3 = 4\lambda, \quad k_2 = 6\lambda^2, \quad k_1 = 4\lambda^3, \quad k_0 = \lambda^4. \tag{52}$$

With $\lambda = 2$, the gains are $k_3 = 8$, $k_2 = 24$, $k_1 = 32$, $k_0 = 16$, and the tracking error converges to zero asymptotically for any initial condition. Larger values of λ produce faster convergence but require larger control inputs.

3.5 Simulation Results

We implement the tracking controller (47) with $\lambda = 2$ for the circular reference trajectory $y^*(t) = [\cos t, \sin t]^T$. The extended state includes the integrator states $x_7 = u_1$ and $x_8 = \dot{u}_1$, initialized at the hover thrust $x_7(0) = 9.8$ and $x_8(0) = 0$. We test three initial conditions without noise to demonstrate convergence from different starting points. Figure 4 starts on the reference trajectory at $(1, 0)$, where the controller achieves accurate tracking within approximately one cycle. Figure 5 starts at $(2, 1)$, offset from the reference, and converges to the circular trajectory after an initial transient of approximately 4 seconds. Figure 6 starts at $(0, 3)$ with nonzero velocity and a small initial roll angle, representing the largest offset; the controller still achieves convergence within 5 seconds. In all cases, the transient duration is consistent with the designed pole location $s = -2$, which predicts error decay proportional to e^{-2t}.

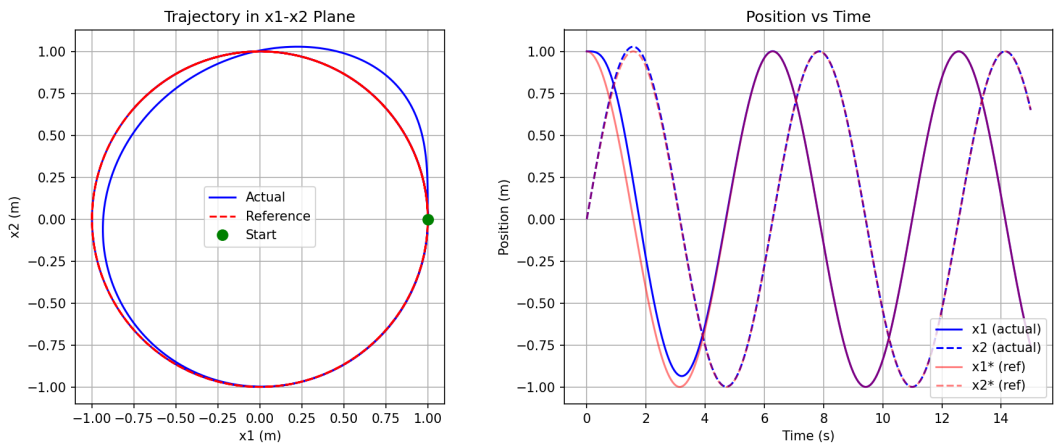

Figure 4: Trajectory tracking from $x_0 = [1, 0, 0, 1, 0, 0, 9.8, 0]^T$ (on the reference). The actual trajectory closely follows the reference circle after a brief initial transient.

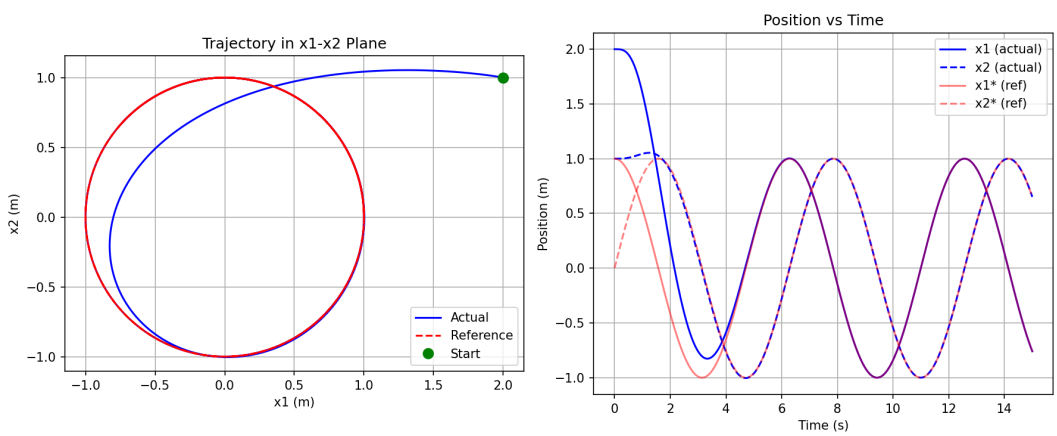

Figure 5: Trajectory tracking from $x_0 = [2, 1, 0, 0, 0, 0, 9.8, 0]^T$ (offset start). The controller drives the multirotor from the offset position to the reference circle within approximately 4 seconds.

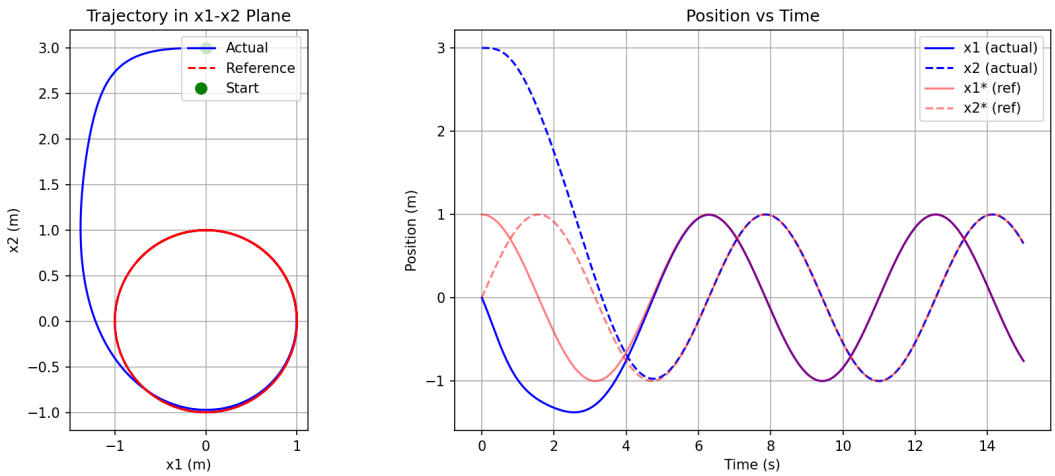

Figure 6: Trajectory tracking from $x_0 = [0,\ 3,\ -1,\ 0,\ 0.1,\ 0,\ 9.8,\ 0]^T$ (far offset with nonzero velocity and roll). Despite the large initial error, the feedback linearizing controller achieves convergence within 5 seconds.

Figure 7 shows tracking under Gaussian process noise ($\sigma = 0.5$ on acceleration states) using Euler integration. The controller maintains stable tracking with small fluctuations around the reference, demonstrating robustness to unmodeled disturbances.

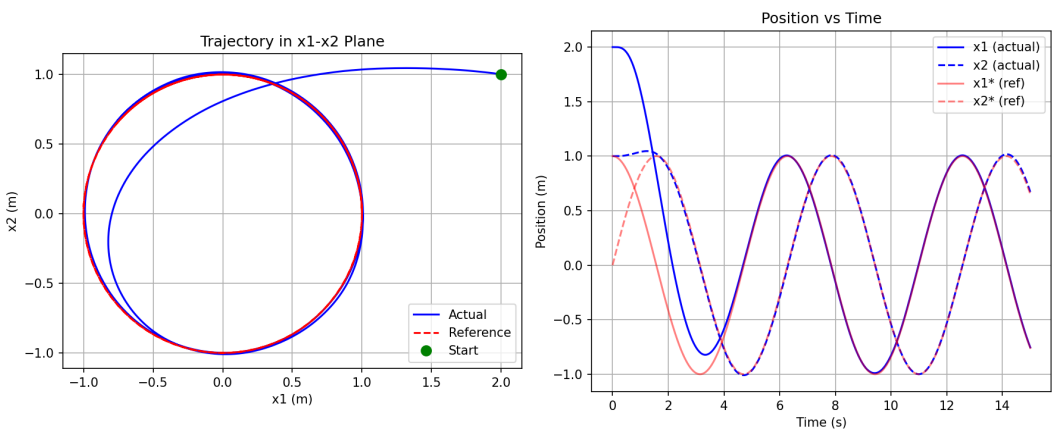

Figure 7: Trajectory tracking with Gaussian process noise ($\sigma = 0.5$). The feedback linearizing controller maintains stable tracking despite disturbances.

4 Part III: Safety from Control Barrier Functions (CBF)

The dynamic feedback linearizing controller from Part II achieves accurate trajectory tracking, but it treats the reference $y^*(t)$ as the sole specification of desired behavior. In practice, the multirotor must also respect spatial constraints imposed by obstacles, walls, or operational boundaries, constraints that the tracking controller has no awareness of. One could attempt to encode these constraints by reshaping the reference trajectory, but this requires solving a trajectory optimization problem at each replanning step and offers no formal guarantee that the constraint will be satisfied between waypoints. Control Barrier Functions provide an alternative approach: rather than modifying the reference, we leave the tracking controller intact and add a safety filter that minimally adjusts the control input whenever the constraint would be violated. This decouples the tracking objective from the safety objective and yields forward invariance of the safe set as a mathematical guarantee, provided the dynamics are known exactly.

4.1 Barrier Function Analysis

We consider the candidate barrier function $h(x) = L - x_1$, which defines the safe set $\mathcal{C} = \{x \mid h(x) \geq 0\} = \{x \mid x_1 \leq L\}$. Geometrically, this constrains the multirotor to remain to the left of the vertical line $x_1 = L$. For h to be a valid CBF, we require $L_g h \neq 0$ so that the input can directly enforce the safety constraint. Computing the time derivative:

$$\dot{h} = -\dot{x}_1 = -v_1. \tag{53}$$

Since $\dot{h}$ depends only on the velocity state v_1 and not on either input, we have $L_{g_1} h = 0$ and $L_{g_2} h = 0$. Therefore, h is not a valid CBF for the original system. Physically, the inputs affect accelerations, not velocities, so a position-only barrier function cannot have relative degree one.

We proceed to check whether a higher-order CBF can be constructed by differentiating again:

$$\ddot{h} = -\dot{v}_1 = u_1 \sin\theta. \tag{54}$$

The input u_1 now appears, but with coefficient $\sin\theta$, which vanishes at the hover equilibrium $\theta^* = 0$. Moreover, u_2 still does not appear. This is the same underactuation issue encountered in Part II: $h = L - x_1$ depends on the first output $y_1 = x_1$, whose relative degree was shown to not be well-defined on the original system.

Since the tracking controller from Part II operates on the extended system (with states $x_{\text{ext}} \in \mathbb{R}^8$ and inputs $[\tilde{u}_1,\, u_2]^T$), we construct the higher-order CBF on this extended system. From the analysis in Section 3.2, $h = L - x_1$ has relative degree 4 on the extended system, with the successive derivatives:

$$\dot{h} = -v_1, \tag{55}$$

$$\ddot{h} = x_7 \sin\theta, \tag{56}$$

$$h^{(3)} = x_8 \sin\theta + x_7 \omega \cos\theta, \tag{57}$$

$$h^{(4)} = \tilde{u}_1 \sin\theta + x_7 u_2 \cos\theta + b_h(x_{\text{ext}}), \tag{58}$$

where $b_h(x_{\text{ext}})$ collects state-only terms, consistent with the structure found in (38). Note that $h^{(4)} = -y_1^{(4)}$, so the input coefficients are the negatives of those in the y_1 channel.

For the higher-order CBF construction, we define a sequence of barrier functions using class-$\mathcal{K}$ functions $\alpha_i(s) = \gamma_i s$ with gains $\gamma_i > 0$:

$$\Psi_0 = h = L - x_1, \tag{59}$$

$$\Psi_1 = \dot{\Psi}_0 + \gamma_1 \Psi_0 = -v_1 + \gamma_1 (L - x_1), \tag{60}$$

$$\Psi_2 = \dot{\Psi}_1 + \gamma_2 \Psi_1. \tag{61}$$

We compute $\dot{\Psi}_1$ explicitly for the extended system. Using (56):

$$\dot{\Psi}_1 = \ddot{h} + \gamma_1 \dot{h} = x_7 \sin\theta - \gamma_1 v_1. \tag{62}$$

Therefore:

$$\Psi_2 = x_7 \sin\theta - \gamma_1 v_1 + \gamma_2\big(-v_1 + \gamma_1 (L - x_1)\big) = x_7 \sin\theta - (\gamma_1 + \gamma_2) v_1 + \gamma_1 \gamma_2 (L - x_1). \tag{63}$$

$$\Psi_3 = \dot{\Psi}_2 + \gamma_3 \Psi_2. \tag{64}$$

The derivative $\dot{\Psi}_2$ involves $h^{(3)}$ and lower-order terms, all of which are state-dependent (no inputs yet). The inputs $[\tilde{u}_1,\, u_2]$ first appear in $\dot{\Psi}_3$, since this involves $h^{(4)}$ from (58). The final CBF enforcement condition is then:

$$\dot{\Psi}_3 + \gamma_4 \Psi_3 \geq 0, \tag{65}$$

which is an affine constraint in the inputs $[\tilde{u}_1,\, u_2]^T$ and can therefore be incorporated as a linear inequality in a quadratic program.

The nested safe sets $\mathcal{C}_i = \{x \mid \Psi_i(x) \geq 0\}$ for $i = 0, 1, 2, 3$ ensure that the original constraint $h(x) \geq 0$ is maintained with sufficient margin at each derivative level. The gains $\gamma_1, \gamma_2, \gamma_3, \gamma_4 > 0$ control how aggressively the barrier is enforced. Larger gains provide stronger safety margins but may restrict the tracking performance more.

4.2 CBF-QP Safety Filter

The tracking controller (47) from Part II produces a nominal input $u_{\text{nom}} = [\tilde{u}_{1,\text{nom}}, u_{2,\text{nom}}]^T$ that achieves asymptotic trajectory tracking but does not account for the safety constraint $x_1 \leq L$. To enforce safety while preserving tracking performance as much as possible, we filter the nominal input through a quadratic program (QP) at each timestep. The CBF condition (65) is affine in the inputs. Writing $\dot{\Psi}_3$ explicitly, it takes the form

$$\dot{\Psi}_3 = L_f \Psi_3(x_{\text{ext}}) + L_g \Psi_3(x_{\text{ext}}) \begin{bmatrix} \tilde{u}_1 \\ u_2 \end{bmatrix}, \tag{66}$$

where $L_f \Psi_3$ collects state-dependent terms and $L_g \Psi_3 \in \mathbb{R}^{1\times 2}$ is the row vector of input coefficients. The CBF constraint (65) then becomes the linear inequality

$$L_g \Psi_3 \begin{bmatrix} \tilde{u}_1 \\ u_2 \end{bmatrix} \geq -L_f \Psi_3 - \gamma_4 \Psi_3. \tag{67}$$

The safety-filtered controller is the solution to the following optimization problem, solved at each timestep:

$$\begin{aligned} u^*(x) = \underset{\mu \in \mathbb{R}^2}{\text{argmin}} \quad & \|\mu - u_{\text{nom}}\|^2 \\ \text{subject to} \quad & L_g \Psi_3 \, \mu \geq -L_f \Psi_3 - \gamma_4 \Psi_3. \end{aligned} \tag{68}$$

This is a QP with a single linear inequality constraint, which admits a closed-form solution. When the nominal input already satisfies the CBF constraint (i.e., the system is far from the boundary $x_1 = L$), the QP returns $u^* = u_{\text{nom}}$ and tracking is unaffected. When the constraint would be violated, the QP projects the nominal input onto the feasible set, applying the minimum correction necessary to maintain safety. This structure ensures that the safety filter intervenes only when needed, preserving the tracking performance of the feedback linearizing controller everywhere except near the safety boundary.

4.3 Simulation Results

We simulate the CBF-based safety controller for the circular reference trajectory $y^*(t) = [\cos t, \sin t]^T$ with barrier $h(x) = L - x_1$ and $L = 0.6$. The HOCBF gains are set to $\gamma_1 = \gamma_2 = \gamma_3 = \gamma_4 = 5$. The initial condition is $x_0 = [0.5, 0, 0, 1, 0, 0, 9.8, 0]^T$, which lies inside the safe set ($x_1 = 0.5 < L = 0.6$).

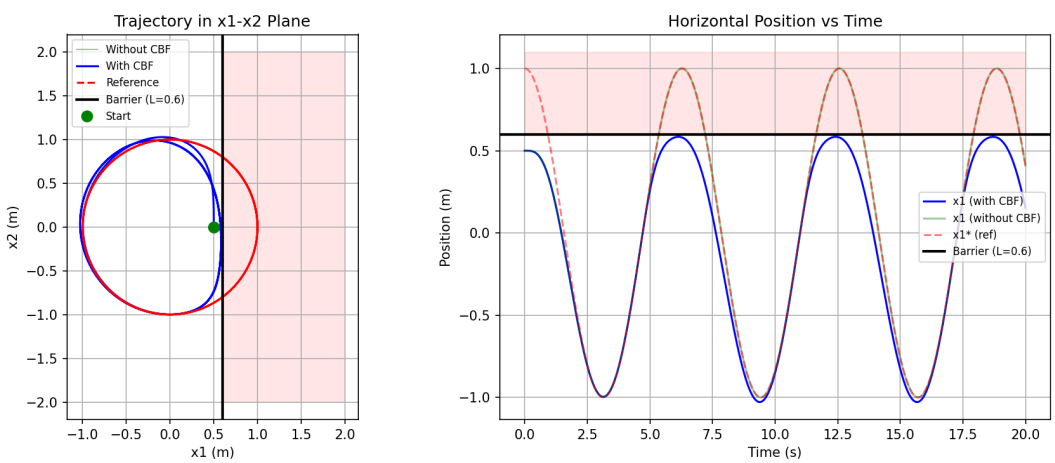

Figure 8: Trajectory tracking with CBF safety filter (no noise). The green trajectory (without CBF) crosses the barrier, while the blue trajectory (with CBF) respects the constraint $x_1 \leq 0.6$. The CBF intervenes only near the boundary, preserving tracking performance elsewhere.

Figure 9 shows the same simulation with Gaussian process noise ($\sigma = 0.3$ on acceleration states). The CBF maintains safety despite the disturbances, with the trajectory remaining within the safe set. Small fluctuations are visible near the barrier, but no violation occurs.

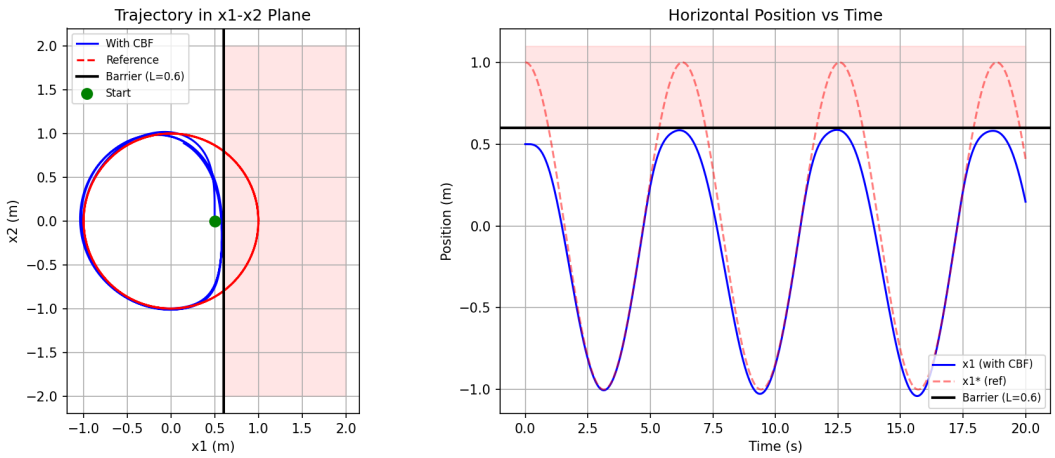

Figure 9: CBF-filtered tracking with Gaussian process noise ($\sigma = 0.3$). The safety constraint is maintained despite stochastic disturbances.

Figure 10 demonstrates the effect of parameter mismatch, where the true system has $m = 1.1$ and $g = 10.0$ while the controller assumes $m = 1.0$ and $g = 9.8$. The safety constraint on x_1 is largely maintained, but the tracking performance degrades significantly, particularly in the vertical direction. Because the true gravity ($g = 10.0$) exceeds the assumed value ($g = 9.8$) by $\Delta g = 0.2$, the nominal thrust under-compensates the actual gravitational force, causing a persistent downward drift. The mass mismatch further weakens the effective control authority, resulting in a distorted trajectory. This highlights a fundamental limitation of the feedback linearization approach, since the controller relies on exact cancellation of the nonlinear dynamics, any model error directly translates into tracking degradation. Robust or adaptive extensions of the CBF framework would be needed to handle such uncertainties with formal guarantees.

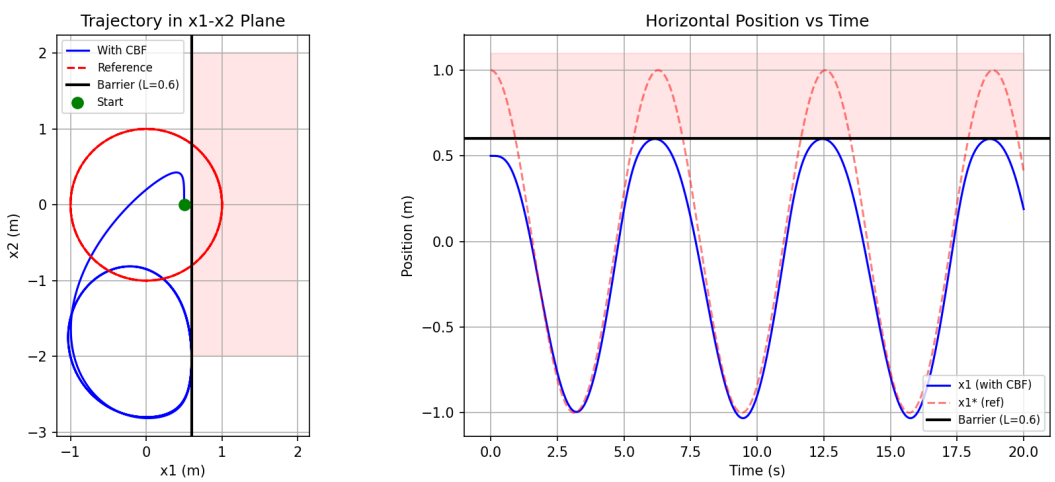

Figure 10: CBF-filtered tracking with parameter mismatch ($m = 1.1$, $g = 10.0$). The $x_1 \leq L$ constraint is approximately maintained, but vertical tracking is significantly degraded due to uncompensated model errors.

4.4 Application to a Figure-Eight Reference

To verify that both the tracking controller and the CBF safety filter generalize beyond the circular reference, we additionally tested the framework on a vertical figure-eight (Lemniscate of Gerono) given by $y_1^*(t) = \sin(2\omega t)$ and $y_2^*(t) = \sin(\omega t) + 1$ with $\omega = 0.5$. This trajectory is more demanding than the circle because its curvature varies along the path and the two outputs evolve at different fundamental frequencies, requiring the controller to coordinate thrust and torque across distinct timescales.

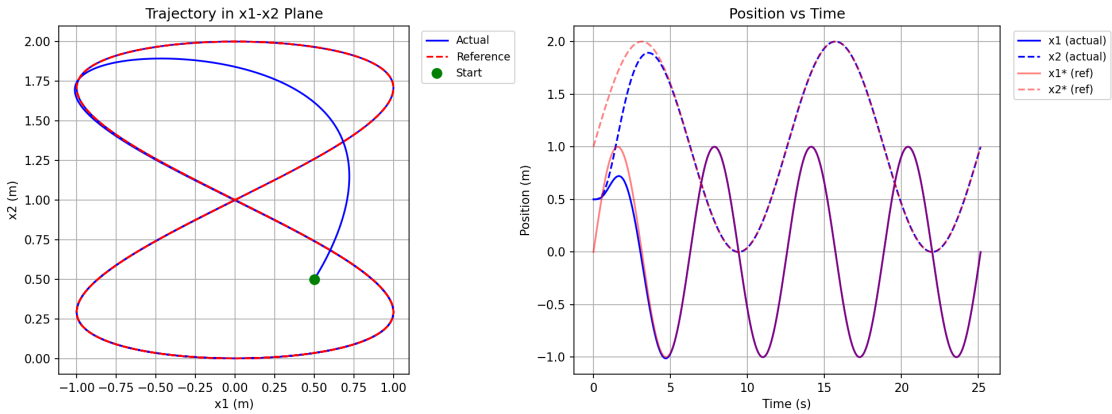

Figure 11: Figure-eight tracking without safety filter, starting from an off-reference initial condition $x_0 = (0.5, 0.5)$ with zero velocity. The controller drives the multirotor onto the reference within roughly one period and tracks accurately thereafter.

Figure 11 shows the nominal tracking controller from Part II following the figure-eight from an off-reference start, demonstrating that the same gains ($\lambda = 2$) used for the circle suffice without retuning. Figure 12 then applies the CBF safety filter with the same barrier $h(x) = L - x_1$ and $L = 0.6$. Because the figure-eight reaches $y_1^* = \pm 1$ at the right turning points, the unfiltered nominal trajectory clearly violates the constraint, while the CBF-filtered trajectory tracks the left lobe accurately and slides along the boundary on the right lobe before returning to the reference once the constraint becomes inactive.

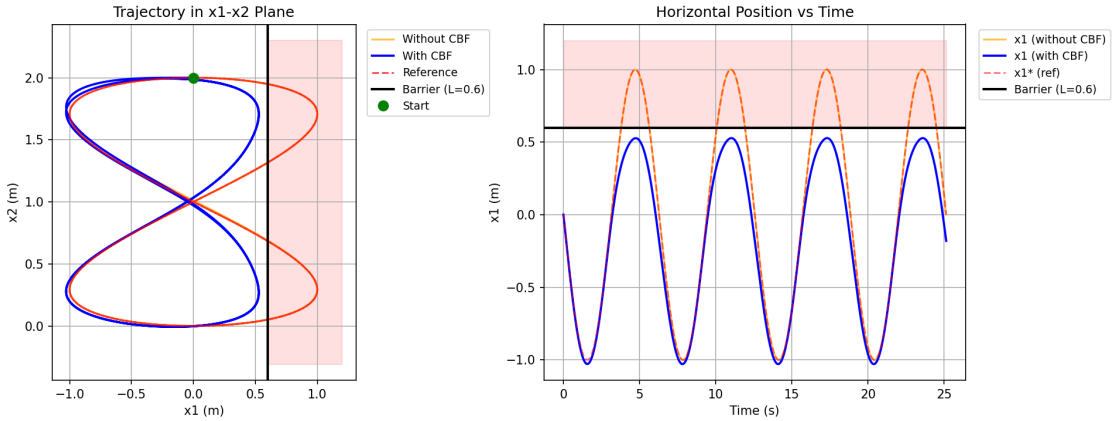

Figure 12: CBF-filtered tracking on the figure-eight reference with $L = 0.6$. The left lobe is tracked accurately, and the right lobe is smoothly diverted at the boundary rather than abruptly clipped, owing to the lower CBF gains ($\gamma_i = 3$) chosen for this experiment.

5 Discussion

5.1 LQR Hover vs. Dynamic Feedback Linearization

The two controllers reflect fundamentally different design philosophies. The LQR hover controller treats the nonlinearity as a disturbance from the equilibrium and accepts a region of validity limited to small perturbations. In return, it inherits the classical robustness margins of LQR: at each input channel of the linearized loop, the controller guarantees a gain margin of $(1/2, \infty)$ and a phase margin of at least $60°$. These margins apply to the linearized closed-loop with respect to gain and phase perturbations at the input, and do not directly translate to robustness against arbitrary parameter variations in the nonlinear plant. However, they suggest a degree of inherent disturbance rejection that is absent in the exact-cancellation structure of feedback linearization. This robustness was visible

in Figure 3, where the hover position was maintained under Gaussian process noise with only small steady-state fluctuations.

The dynamic feedback linearizing controller, by contrast, attempts to cancel the nonlinearity exactly through the coordinate transformation (45) and the input inversion (47). This yields global tracking performance for any feasible reference trajectory, not just regulation about a fixed point. The price is a loss of inherent robustness. Because the controller relies on exact cancellation of $b(x)$ and $A_{\text{ext}}^{-1}(x)$, any model error propagates directly into the tracking error. The parameter-mismatch experiment in Figure 10, where a 10% error in mass and a 2% error in gravity produced visible vertical drift, illustrates this fragility. The LQR controller, lacking the exact-cancellation structure of feedback linearization, would likely be less sensitive to such mismatches in the sense that the linear feedback gain K does not amplify model errors through an inversion. However, without integral action the LQR would still exhibit a steady-state offset under gravity mismatch. A direct simulation comparison under the same mismatch conditions is left for future work.

A second contrast lies in the structure of the control law. LQR is a static feedback $u = u^* + K(x - x^*)$, which can be implemented in a single matrix multiplication. The DFL controller requires evaluating the eight-dimensional coordinate transformation, computing the decoupling matrix and its inverse, and integrating the auxiliary states x_7 and x_8 at every timestep. The added computational cost is modest at the simulation rate used here, but it scales unfavorably with system dimension and would become a consideration in real-time embedded implementations.

5.2 Tracking Performance vs. Safety Enforcement

The CBF-QP layer added in Part III does not replace the tracking controller. It filters the nominal input through a quadratic program that intervenes only when the safety constraint would otherwise be violated. This separation is reflected in the simulation results. In Figure 8, the trajectory closely follows the reference circle wherever x_1 is far from the boundary $L = 0.6$. The deviation appears only in the right portion of the orbit, where the unfiltered controller would drive x_1 past the limit. The CBF clips the trajectory at the boundary and releases the multirotor back to the reference once the constraint becomes inactive.

The cost of this safety layer is a quantifiable degradation in tracking performance near the barrier. At the boundary, the CBF effectively projects the nominal acceleration onto the half-space defined by the linear inequality in (68), which generally produces an input different from u_{nom}. In our simulation, this manifests as a flattening of the right side of the reference circle. For applications where the nominal trajectory is known in advance to violate constraints, a planner-level reshaping might yield smoother behavior. The CBF approach is most useful when constraints are uncertain a priori, when the reference is generated reactively, or when formal forward-invariance guarantees are required.

A subtle but important point is that the CBF inherits the model dependence of the underlying DFL controller. The Lie derivatives $L_f\Psi_3$ and $L_g\Psi_3$ in the CBF constraint are computed using the nominal dynamics, so under parameter mismatch the constraint itself is mis-specified. Figure 10 shows that the safety constraint is approximately maintained even under mismatch, but this is not a guarantee as with larger parameter errors or more aggressive trajectories, the CBF could fail to enforce safety. Robust CBF formulations that explicitly account for bounded model uncertainty would be needed for hard safety guarantees in the presence of disturbances.

6 Conclusion

This report developed three controllers of increasing capability for the planar multirotor: an LQR hover controller from a linearized model, a dynamic feedback linearizing tracking controller on a dynamically extended system, and a CBF-based safety filter applied on top of the tracking controller. Each design choice was motivated by a specific limitation of the preceding approach: the LQR controller cannot track trajectories or operate far from equilibrium, the DFL controller has no awareness of spatial constraints, and the CBF filter inherits the model-dependence of the underlying DFL controller. The simulation results confirm the expected behavior of each controller and quantify the tradeoffs discussed in Section 5.

Several extensions follow naturally from this analysis. Robust or adaptive CBF formulations would address the model-mismatch fragility observed in Figure 10, providing formal safety guarantees under bounded uncertainty. Replacing the LQR or DFL nominal controller with a learning-based policy, while retaining the CBF as a safety filter, would offer a path toward combining data-driven performance with formal safety. Finally, the planar multirotor abstracts away the rotational dynamics of a full quadrotor, extending the design to the six-degree-of-freedom case introduces a non-trivial inner-loop attitude controller and richer constraint geometries.